PROJECT REVIEW 2 REPORT

Course: BCSE408L — Cloud Computing and Full Stack Application Development

Project Title:	Project 12 — CreatorOps (Marketing SaaS Application)
Candidate Name:	Kathyayani T T
GitHub Repository:	https://github.com/ttkathyayani/CreatorOps
Live Public URL:	https://creatorops-rho.vercel.app
Evaluation Stage:	Project Review 2

1. SYSTEM ARCHITECTURE

CreatorOps is architected as a three-tier web application structured around the Model-View-Template (MVT) pattern and RESTful application programming interfaces.

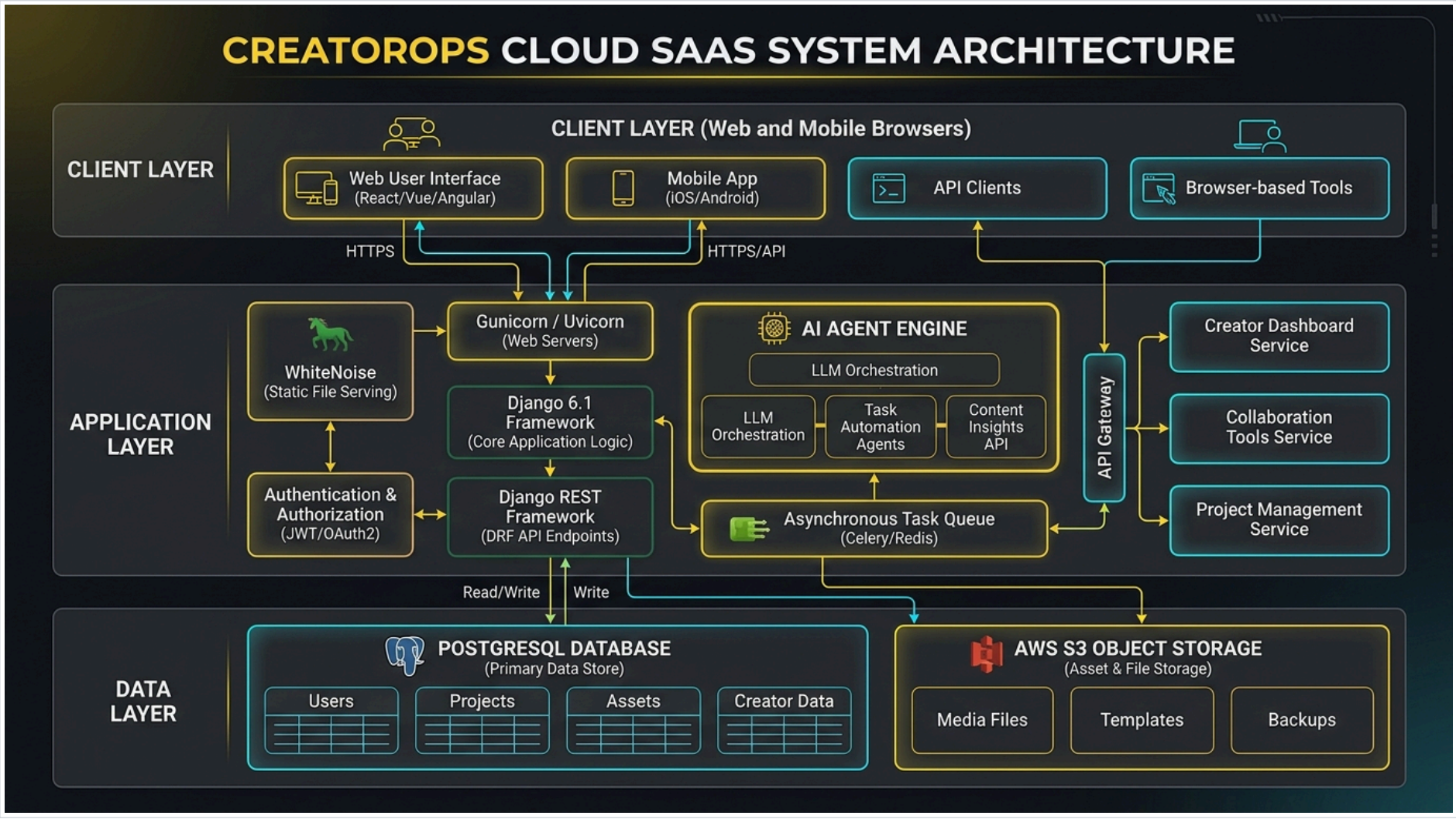


Figure 1: High-Level Three-Tier Architecture of CreatorOps

1.1 Client Tier (Presentation)

The client interface is rendered through responsive server-side HTML5 templates utilizing CSS variables for styling. It provides operational screens for workspace configuration, client brand records, marketing campaigns, content drafting, and approval queues. External clients and automated tools interact via JSON-based REST endpoints over HTTPS.

1.2 Application Tier (Business Logic)

The application tier is powered by Python 3.12 and Django 6.1. It manages routing, tenant isolation, session validation, and content state transitions. A Django REST Framework (DRF) layer exposes ModelViewSets for programmatic operations. In addition, an internal rule-based agent module (`agents/`) provides automated checks on creator scripts—measuring word count, estimated speaking duration, hook keyword presence, and brand safety compliance. WhiteNoise serves static assets directly from the WSGI application layer.

1.3 Data Tier (Persistence)

Persistent relational data is stored in PostgreSQL for production deployment, with SQLite supported for local development. Binary assets (images, documents, and videos) are managed with support for Amazon S3 object storage via `boto3` and `django-storages`.

2. CLOUD DEPLOYMENT ARCHITECTURE

The production application is hosted on Vercel's cloud infrastructure, taking advantage of its edge routing network and serverless execution capabilities.

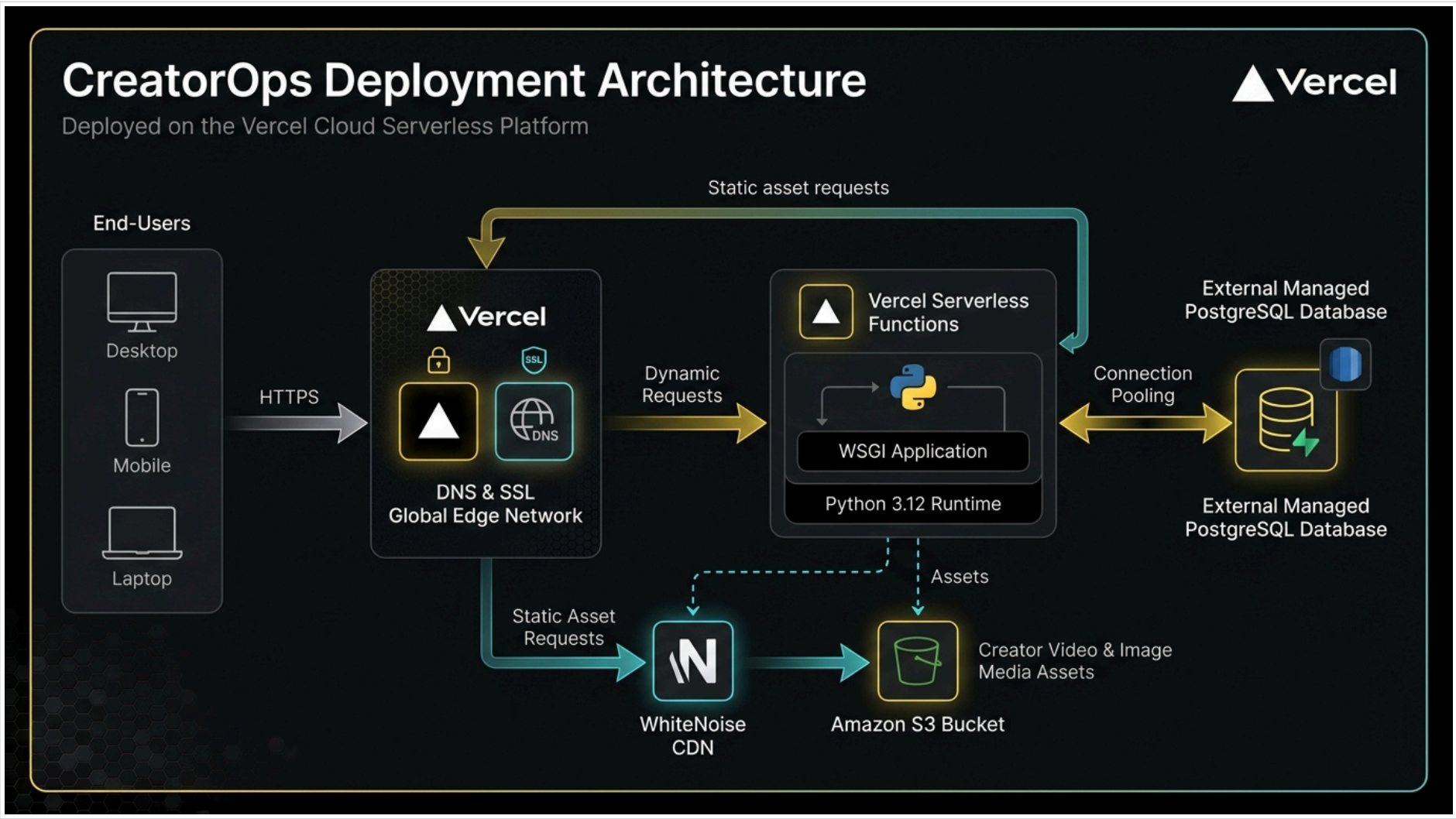


Figure 2: Cloud Deployment Model on Vercel Serverless Edge Platform

2.1 Ingestion and Execution Pipeline

- **DNS and Edge Ingestion:** Traffic is routed through Vercel's Anycast edge network with automatic TLS/SSL termination.
- **Serverless Execution:** Incoming requests execute within Python 3.12 serverless functions via the standard WSGI interface (`config/wsgi.py`).

- **Static Asset Offloading:** Bundled CSS, JavaScript, and diagrams are cached and served from Vercel's content delivery network points of presence.
- **Database Connectivity:** The serverless runtime connects to a managed PostgreSQL database via connection pooling using the `DATABASE_URL` environment variable.

3. DATABASE SCHEMA DESIGN

The relational schema is normalized to Third Normal Form (3NF) and contains nine core entities structured around multi-tenant workspace isolation.

Entity	Primary Key	Foreign Keys	Key Attributes	Integrity Constraints
Workspace	`id`	None	`name`, `slug`, `description`, `created_at`	Unique on `slug`
Membership	`id`	`user_id`, `workspace_id`	`role` (Admin, Manager, Creator, Client)	Unique on `(user, workspace)`
Client	`id`	`workspace_id`	`name`, `contact_email`, `brand_guidelines`	Unique on `(workspace, name)`
Campaign	`id`	`client_id`	`title`, `objective`, `budget`, `status`	Budget non-negative
ContentItem	`id`	`campaign_id`, `creator_id`	`title`, `platform`, `script_body`, `status`	Status state machine enum
ContentVersion	`id`	`content_item_id`, `created_by`	`revision_number`, `script_snapshot`	Unique on `(item, rev_num)`
Approval	`id`	`content_item_id`, `reviewer_id`	`decision`, `comment`, `timestamp`	Decision choices
Asset	`id`	`content_item_id`, `uploaded_by`	`file`, `filename`, `size_bytes`	File extension check
CampaignMetric	`id`	`campaign_id`	`metric_name`, `metric_value`, `recorded_date`	Indexed on recorded date

4. ENTITY-RELATIONSHIP (ER) DIAGRAM

The entity-relationship diagram below maps the tables and cardinalities across the system:

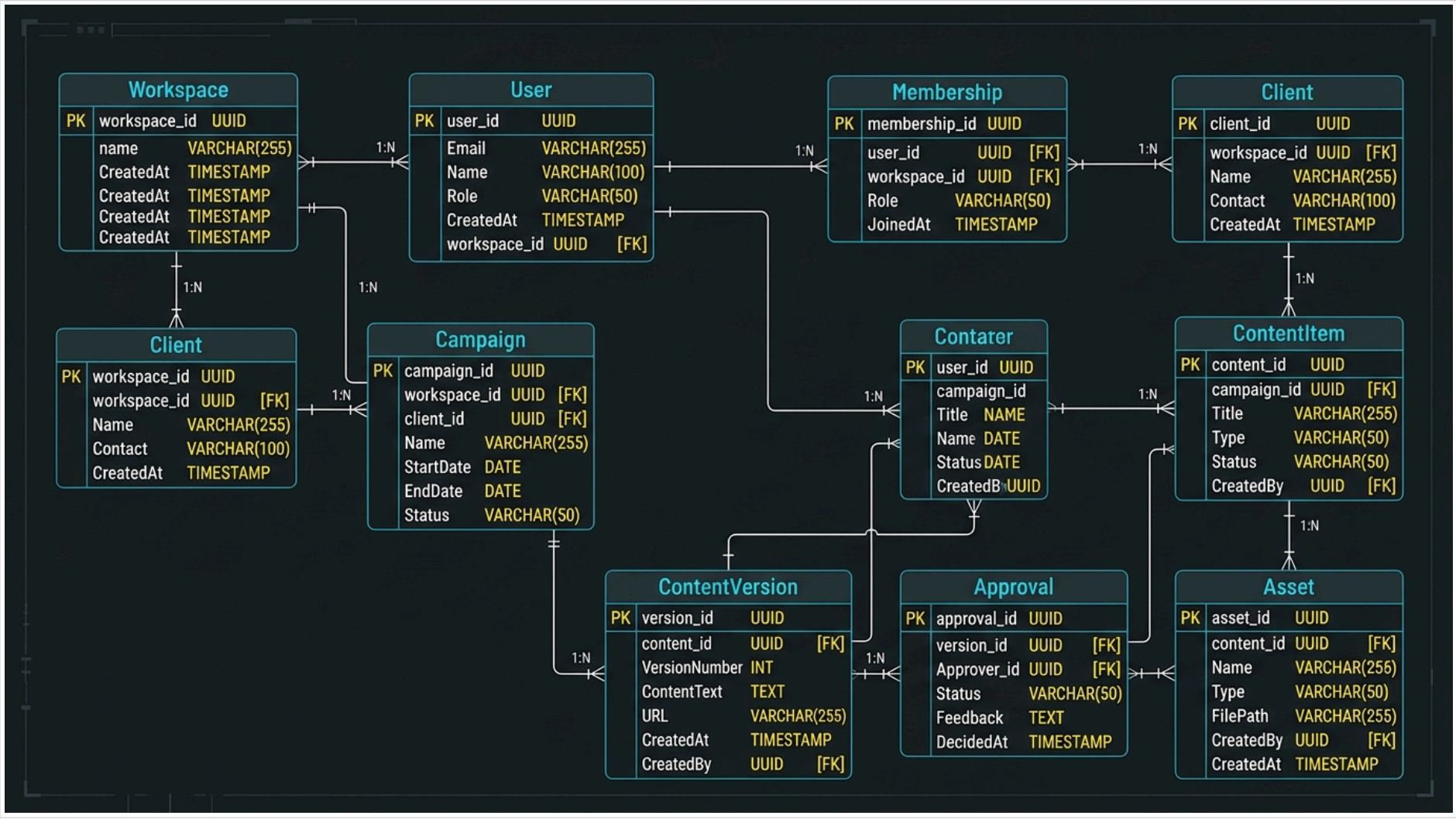


Figure 3: Entity-Relationship Diagram of the CreatorOps Database

4.1 Cardinality Overview:

- **Workspace to Membership:** One-to-Many ($1 : N$). A workspace maintains multiple user memberships.
- **Workspace to Client:** One-to-Many ($1 : N$). A workspace holds multiple client brand profiles.
- **Client to Campaign:** One-to-Many ($1 : N$). A client sponsors multiple marketing campaigns.
- **Campaign to ContentItem:** One-to-Many ($1 : N$). A campaign contains multiple content deliverables.
- **ContentItem to ContentVersion:** One-to-Many ($1 : N$). Each edit generates a sequential revision record.
- **ContentItem to Approval:** One-to-Many ($1 : N$). An item records internal manager and client review decisions.

5. MODULAR ARCHITECTURE

The software follows a domain-driven structure, separated into distinct Django apps:

- `accounts`: Authentication, session management, user registration, and profile data.
- `workspaces`: Multi-tenant organizational boundaries, member roles, and tenant context resolution.
- `clients`: Client brand profiles, contact details, and brand voice guidelines.
- `campaigns`: Marketing campaign objectives, budget allocations, and status tracking.
- `content`: Deliverable management, social platform targeting, and revision snapshotting.
- `approvals`: Two-tier review workflow (Manager Gate followed by Client Sign-off).
- `assets`: Media asset handling and cloud storage integration.
- `analytics`: Dashboard KPI aggregation and performance reporting.
- `agents`: Automated script auditing, hook suggestions, and comment clustering.
- `config`: Root URL routing, project settings, and WSGI entrypoint.

6. REST API DESIGN

The REST API is implemented using Django REST Framework and requires authentication for all protected endpoints:

HTTP Method	Endpoint	Description	Permitted Roles
`GET / POST`	`/api/clients/`	List or create clients in active workspace	Authenticated Member
`GET / PUT / DELETE`	`/api/clients/{id}/`	Retrieve, update, or delete a client profile	Admin, Manager
`GET / POST`	`/api/campaigns/`	List or create marketing campaigns	Authenticated Member
`GET / PUT / DELETE`	`/api/campaigns/{id}/`	Retrieve, update, or delete a campaign	Admin, Manager
`GET / POST`	`/api/content/`	List or create content deliverables	Authenticated Member
`GET / PUT / DELETE`	`/api/content/{id}/`	Retrieve, update, or delete content item	Creator, Manager
`POST`	`/api/content/{id}/review/`	Submit manager or client approval decision	Manager, Client
`GET`	`/api/reports/campaigns/{id}/`	Retrieve campaign metrics summary	Authenticated Member
`POST`	`/api/agents/script-review/`	Run automated script quality and brand compliance audit	Creator, Manager

7. AUTHENTICATION AND ROLE-BASED ACCESS CONTROL

The system implements a four-tier Role-Based Access Control (RBAC) model integrated with multi-tenant data boundaries:

- **Password Protection:** Passwords are stored as salted PBKDF2-SHA256 hashes.
- **Session Isolation:** Sessions are validated with CSRF protection on all state-modifying requests.
- **Tenant Boundaries:** The active workspace is resolved per request. Database operations automatically filter by workspace ID to ensure tenant data isolation.

- **Role Definitions:**
  1. **Admin:** Global control over workspace settings, members, and roles.
  2. **Manager:** Client brand setup, campaign creation, and primary draft approvals.
  3. **Creator:** Content drafting, script versioning, and asset uploads.
  4. **Client Reviewer:** Read-only access to campaign deliverables with final approval sign-off.

8. CORE MODULES IMPLEMENTATION SUMMARY

- **Content Lifecycle State Machine:** Programmatically manages content transitions across 7 stages: `draft` → `pending_manager` → `pending_client` → `approved` → `changes_requested` → `scheduled` → `published`.
- **Revision Snapshots:** Every script or caption change generates an immutable `ContentVersion` record, maintaining a complete audit trail.
- **Automated Script Analysis:** The script review module checks text against word count limits, spoken duration pacing, hook keyword placement in the opening sentence, and forbidden brand terms.

9. GITHUB REPOSITORY

The complete project source code is maintained on GitHub under continuous version control:

Repository:	https://github.com/ttkathyayani/CreatorOps
Default Branch:	`main`

10. CLOUD DEPLOYMENT VERIFICATION

The application is actively deployed and serving production traffic on Vercel:

Deployment Metric	Value	Status
Hosting Environment	Vercel Serverless Edge Platform	Active
Production URL	https://creatorops-rho.vercel.app	HTTP 200 OK
SSL/TLS Certificate	Let's Encrypt Wildcard Edge Certificate	Verified
Static Delivery	WhiteNoise 6.9 + Vercel Edge Cache	Operational